A REPORT ON
SUMMER INTERNSHIP

DIYA: DESIGN AND DEVELOPMENT OF AN UNATTENDED MEDITATION KIOSK

Kiosk application, phone-based visitor identification and hardware pipeline integration

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

Submitted by

AYUSH GUPTA

Registration No.: [Registration No.]

Internship carried out at

INDReA Lab, Department of Design

Indian Institute of Technology Hyderabad

1 June 2026 – 31 July 2026

Under the supervision of

Dr. Prasad S. Onkar
Associate Professor, Department of Design
Indian Institute of Technology Hyderabad

Faculty guide

[Faculty guide name]
[Designation]
Department of Computer Science and Engineering

University logo

Insert the university logo here

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MANIPAL UNIVERSITY

[Campus / Institute, e.g. Manipal University Jaipur]

October 2026

CERTIFICATE FROM THE ORGANISATION

PLACE SCANNED CERTIFICATE HERE

Insert the scanned internship certificate here (signed by Dr. Prasad S. Onkar, Department of Design, IIT Hyderabad, dated 07/10/2026). Replace this whole box with the scan, scaled to the page width.

DECLARATION

I, **Ayush Gupta**, a student of Bachelor of Technology in Computer Science and Engineering at Manipal University, hereby declare that the internship report titled **"Diya: Design and Development of an Unattended Meditation Kiosk"** is a record of the work carried out by me during my summer internship at the INDReA Lab, Department of Design, Indian Institute of Technology Hyderabad, from 1 June 2026 – 31 July 2026, under the supervision of Dr. Prasad S. Onkar.

The work presented in this report is original and has not been submitted, in part or in full, for the award of any other degree or diploma. Work done by the collaborating hardware and computer-vision team is described only where it is needed to explain the system, and is acknowledged as theirs wherever it appears.

(Signature)
Ayush Gupta
Registration No.: [Registration No.]
Date: [dd/mm/yyyy]
Place: [Place]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr. Prasad S. Onkar**, Associate Professor, Department of Design, Indian Institute of Technology Hyderabad, for giving me the opportunity to work at the INDReA Lab and for his guidance and encouragement throughout the internship.

I am deeply thankful to **Mr. Eshwar Teja**, under whose day-to-day guidance this project was carried out, for the many design discussions, for testing every build on the real kiosk, and for his patience while the system was taking shape. I also thank **Kalyan Sir** for his support and valuable feedback during the internship.

I thank the members of the hardware and computer-vision team, whose camera, motor and meditation software the kiosk application integrates with, for their cooperation in defining clear interfaces between our parts of the system.

I am grateful to [Faculty guide name], my faculty guide, and to the Department of Computer Science and Engineering, Manipal University, for their support and for making this internship possible. Finally, I thank my family and friends for their constant encouragement.

(Signature)

Ayush Gupta

ABSTRACT

Diya is an unattended meditation kiosk developed at the INDReA Lab, Department of Design, IIT Hyderabad, and named after the traditional Indian oil lamp. A visitor walks up to the kiosk, is identified, is physically aligned to a set of cameras by a motorised rig, sits through a guided meditation while thermal, depth, gaze and posture data are captured, and receives a personalised report on screen. The machine then resets itself for the next person, with no operator present.

This report describes the software I designed and built for the kiosk during my eight-week summer internship: a full-screen desktop application written in C# on .NET 8 with the Avalonia UI framework for Ubuntu (GNOME on Wayland); a small Node.js/Express and PostgreSQL backend hosted on Render; the phone-facing web pages used for registration, roster administration and login; the integration layer that launches the hardware team's camera and meditation pipeline and displays the PDF report it produces; and the packaging that ships the kiosk as a self-contained Debian package that starts on boot and restarts after a crash.

The most significant design decision was to **invert the usual QR-code flow**. The kiosk has no scanner, so instead of the visitor presenting a code, the kiosk displays a short-lived session QR code and the visitor's own phone scans it using a private, per-person login link. This removed a hardware dependency, needed no app installation, and turned the scan into proof of *who* arrived. The second important decision was to integrate with the hardware team through a process and a file on disk, rather than an API, which let the two teams work independently.

The system went through seven released versions (v1.0 to v1.6) in eight weeks and was demonstrated end to end with a real phone scanning the kiosk. The report also documents the problems encountered, in particular the window-management restrictions of Wayland, and the work that remains: a consent screen, an inactivity reset, continuous-integration builds, and replacing the stub calibration scripts with the hardware team's real motor-control stages.

Keywords: kiosk, meditation, QR code, Avalonia, .NET 8, Wayland, Node.js, PostgreSQL, human–computer interaction, systems integration.

TABLE OF CONTENTS

Certificate from the Organisation . . . . . **ii**
Declaration . . . . . **iii**
Acknowledgement . . . . . **iv**
Abstract . . . . . **v**
List of Figures . . . . . **viii**
List of Tables . . . . . **ix**
List of Abbreviations . . . . . **x**
1 Introduction **1**
1.1 About the Organisation 1
1.2 Project Background 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope of the Work 3
1.6 Organisation of the Report 3
2 Background and Technology Study **4**
2.1 Requirements of an Unattended Kiosk 4
2.2 Approaches to Visitor Identification 4
2.3 Choice of Desktop UI Framework 4
2.4 Linux Desktop under Wayland 5
2.5 Tools and Technologies Used 6
3 Requirements and System Design **7**
3.1 Functional Requirements 7
3.2 Non-Functional Requirements 7
3.3 System Architecture 8
3.4 Visitor Flow 8
3.5 Interaction Sequence 9
3.6 Data Design 10
3.7 API Design 11
3.8 User Interface Design 12
4 Kiosk Apparatus and Hardware Setup **15**
4.1 Overview of the Apparatus 15
4.2 Hardware Components 15

4.3 Hardware Block Diagram 16
4.4 Photographs of the Apparatus 16
4.5 Software–Hardware Boundary 19
5 Implementation 21
5.1 Structure of the Kiosk Application 21
5.2 Fullscreen Kiosk Shell on Wayland 21
5.3 Session QR, Polling and the Home Screen 22
5.4 Backend Service 22
5.5 Web Pages for Admin and Phone 23
5.6 Pipeline Integration and Report Display 24
5.7 Packaging and Deployment 25
5.8 Screens of the Final Application 26
6 Evolution of the Design 29
6.1 Iterative Development 29
6.2 From Data in the QR to a Session Token 30
6.3 Inverting the QR Flow 30
6.4 Alternatives That Were Explored 30
6.5 Window Hand-off Experiment and Rollback 30
7 Testing and Results 31
7.1 Testing Approach 31
7.2 Test Cases 31
7.3 Testing Without Cameras 32
7.4 Field Demonstration 32
7.5 Results 33
8 Challenges and Learnings 35
8.1 Challenges Faced 35
8.2 Learnings 35
9 Conclusion and Future Work 37
9.1 Conclusion 37
9.2 Future Work 37
References 38
Appendix A Weekly Work Log 39
Appendix B Installation and Operation Commands 41
Appendix C Repository Structure 42

LIST OF FIGURES

3.1 Overall system architecture of the Diya kiosk 8
3.2 Visitor flow as a state machine 9
3.3 Sequence of interactions between the admin, phone, kiosk, backend and meditation-app 10
3.4 Database schema (entity–relationship view) 11
3.5 Wireframe WF-01: kiosk idle, waiting to be scanned 12
3.6 Wireframe WF-02: visitor identified, session starting 13
3.7 Wireframe WF-03: report overlay with the single Return action 13
3.8 Wireframe WF-04: the visitor's three taps on the phone 14
4.1 Hardware block diagram of the kiosk 16
4.2 Front view of the Diya meditation kiosk 17
4.3 Side view of the kiosk showing the seat and sensor arm 17
4.4 Camera assembly (C920, RealSense D435, Arducam, thermal camera) 18
4.5 Motorised chest and eye axes with the actuator mechanism 18
4.6 Arduino controller and wiring 19
4.7 LD2410 radar presence sensor and its mounting 19
5.1 Web pages of the system (sample data) 24
5.2 Retry logic of the pipeline script run1.sh 25
5.3 Build and deployment of the kiosk and the backend 26
5.4 Kiosk idle: session created and QR code displayed 27
5.5 Kiosk after the phone has claimed the session 27
5.6 Pipeline running, with the live status line 28
5.7 Report displayed in the app with the thank-you message and Return 28
6.1 Timeline of the main versions 29
7.1 Live demonstration: the visitor's phone 33
7.2 Live demonstration: the kiosk after the phone has claimed the session (email address blurred) 33
A.1 Gantt chart of the internship (1 June – 31 July 2026) 39

LIST OF TABLES

1.1 Division of responsibilities between the collaborating parties 1
2.1 Comparison of visitor identification approaches 4
2.2 Comparison of desktop UI frameworks for the kiosk 4
2.3 Tools and technologies used in the project 6
3.1 Functional requirements 7
3.2 Non-functional requirements 7
3.3 REST endpoints of the backend 11
4.1 Hardware components of the kiosk 15
5.1 Modules of the kiosk application 21
5.2 Configuration through environment variables 26
6.1 Version history 29
7.1 Summary of test cases 31
8.1 Main challenges and how they were addressed 35
A.1 Weekly work log 39

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CI	Continuous Integration
CV	Computer Vision
DB	Database
.deb	Debian software package format
E2E	End-to-End
GNOME	GNU Network Object Model Environment (Linux desktop)
HTTPS	Hypertext Transfer Protocol Secure
IITH	Indian Institute of Technology Hyderabad
JSON	JavaScript Object Notation
LTS	Long-Term Support
PDF	Portable Document Format
PR	Pull Request
QR	Quick Response (two-dimensional barcode)
REST	Representational State Transfer
SQL	Structured Query Language
UI / UX	User Interface / User Experience
V4L2	Video4Linux version 2 (Linux camera interface)
XLSX	Office Open XML spreadsheet (Excel) format

CHAPTER 1
INTRODUCTION

1.1 About the Organisation

The **Indian Institute of Technology Hyderabad (IITH)** is one of the second-generation IITs, established in 2008 and located at Kandi, Sangareddy, Telangana. Alongside its engineering and science departments, IITH has a **Department of Design** that works across product, interaction and communication design and frequently combines design research with engineering prototypes.

The internship was carried out at the **INDReA Lab** in the Department of Design, under the supervision of Dr. Prasad S. Onkar, Associate Professor, from 1 June 2026 to 31 July 2026. The lab was developing a meditation kiosk inspired by the *diya*, and I joined the project to build its software.

1.2 Project Background

A *diya* is a small oil lamp that has been part of Indian homes and rituals for centuries. A steady flame is a familiar aid to concentration, and the project takes the lamp as its inspiration: a calm, self-contained object that invites a person to pause.

The Diya kiosk turns this idea into an installation meant for a public, museum-like space. A visitor walks up, is identified, and is guided through a short meditation. During the session several cameras observe the person (a webcam for face and gaze, a depth camera, a posture camera and a thermal camera), and a motorised rig first aligns these sensors to the visitor's chest and eyes. At the end, a report summarising the session is shown on screen. The kiosk then returns to its welcome screen, ready for the next visitor, with **no attendant**.

Three groups contributed to the system. I was responsible for the visitor-facing kiosk software, the visitor identification system, the integration layer and the packaging. A collaborating hardware and computer-vision team built the camera and motor scripts and a separate `meditation-app` that runs the session and writes the report. Table 1.1 summarises the split.

Table 1.1: Division of responsibilities between the collaborating parties

Party	Owns	Interface to the kiosk app
Me (this internship)	Kiosk desktop app, backend API and database, web pages for registration/admin/login, pipeline launcher, report display, packaging and deployment	—
Hardware / CV team	Camera calibration and motor scripts (HOME, SHOOT, CHEST, EYE), the `meditation-app` package that runs the session and generates the PDF report	Shell script `run1.sh` and the folder `/opt/meditation-app/data`
Lab / supervisors	Concept, experience design, hardware rig, testing on the real kiosk	Requirements and review

1.3 Problem Statement

An unattended kiosk has to do, on its own, what an attendant would normally do: greet the visitor, find out who they are, start the right process, recover when something fails, and get ready for the next person. For Diya this had to work on a Linux machine with several cameras attached, alongside software written by another team, and without a QR scanner or keyboard for the visitor. The problem addressed in this internship was therefore:

To design and build reliable, self-recovering kiosk software that identifies a visitor without dedicated hardware, runs the hardware team's calibration and meditation pipeline, shows the resulting report, and resets automatically for the next visitor, while remaining easy to install and update on the kiosk machine.

1.4 Objectives

1. Build a full-screen kiosk application for Ubuntu (GNOME on Wayland) that starts on boot, cannot easily be left by a visitor, and restarts itself after a crash.
2. Design a visitor identification method that needs no scanner or keyboard at the kiosk.
3. Build a small backend and web pages for visitor registration, roster upload by an administrator, and phone-based login.
4. Integrate the hardware team's calibration scripts and meditation application without depending on their internal code.
5. Display the generated PDF report inside the kiosk and reset for the next visitor.
6. Package the application so it installs on a fresh machine with a single command, and document installation, operation and troubleshooting.

1.5 Scope of the Work

The work covered the complete software path that a visitor experiences, from the welcome screen to the report, together with the backend, the web pages and the deployment. Building the camera and motor control logic and the meditation content was **outside the scope** of this internship; these were provided by the hardware team. They are described in Chapter 4 because the kiosk software had to be designed around them.

1.6 Organisation of the Report

- **Chapter 2** reviews the background and explains the choice of technologies.
- **Chapter 3** lists the requirements and presents the system design.
- **Chapter 4** describes the kiosk apparatus and its hardware.
- **Chapter 5** explains the implementation of each part of the software.
- **Chapter 6** traces how the design evolved across its versions.
- **Chapter 7** covers testing and the results of the demonstration.
- **Chapter 8** discusses the main challenges and what I learned.
- **Chapter 9** concludes the report and lists future work.
- **Appendices** contain the weekly work log, setup commands and the repository structure.

CHAPTER 2
BACKGROUND AND TECHNOLOGY STUDY

2.1 Requirements of an Unattended Kiosk

Kiosk software differs from an ordinary desktop application mainly in what happens when nobody is watching. It must start without a login prompt, occupy the whole screen, survive crashes and power cuts, prevent a visitor from reaching the desktop, and return to a known state after every visitor. Every failure must either be recovered automatically or be shown clearly enough that a member of staff can act on it. Because the kiosk also uses cameras and stores personal details, privacy is a requirement in its own right.

2.2 Approaches to Visitor Identification

Several ways of identifying a visitor were considered. Table 2.1 compares them against the constraints of this kiosk: no scanner, no keyboard for visitors, and a public setting.

Table 2.1: Comparison of visitor identification approaches

Approach	Hardware at kiosk	Visitor effort	Remarks
Printed pass + USB QR scanner	QR scanner	Carry and present a pass	Needs extra hardware and printed passes; explored as an offline variant
Typing name / email at kiosk	Keyboard or on-screen keyboard	High	Kept only as a fallback for staff and testing
Visitor shows QR from phone to kiosk camera	Camera pointed at visitor	Medium	The kiosk cameras are needed by the CV pipeline
Kiosk shows QR, phone scans it	None (just the display)	Low: open link, tap, scan	**Chosen.** Uses the camera the visitor already owns
Face recognition	Camera	None	Strong option for the future; requires enrolment and careful privacy handling

2.3 Choice of Desktop UI Framework

The kiosk application has to run fullscreen on Linux, launch native processes, read files written by other programs, and stay up all day. Table 2.2 compares the options that were considered.

Table 2.2: Comparison of desktop UI frameworks for the kiosk

Option	Linux support	Native process / hardware access	Verdict
C# / .NET 8 + Avalonia	Official, renders with Skia on X11 and Wayland	Full (.NET Process, file system)	**Chosen.** Managed memory, single self-contained binary
.NET MAUI	Not officially supported on Linux	Full	Rejected: no Linux target
Electron	Good	Needs a native helper (sidecar)	Heavier; still needs a second process
Pure web app in a browser	Good	None from the browser sandbox	Cannot launch the local pipeline
C++ with Qt	Good	Full	No real performance benefit for this UI; manual memory management

C# with **.NET 8 (LTS)** gives garbage-collected memory, which removes a whole class of crashes for a machine that runs all day. **Avalonia 11** is a cross-platform .NET UI framework that officially supports Linux and draws its own controls with Skia, so the interface looks the same on the development laptop and on the kiosk.

2.4 Linux Desktop under Wayland

The kiosk runs Ubuntu with the GNOME desktop on **Wayland**. Unlike the older X11 system, a Wayland compositor (here, Mutter) decides where windows go, and applications are deliberately not allowed to move, raise or hide other applications' windows, or even to raise themselves. This is good for security, but it affects a kiosk directly: tricks that work on X11, such as forcing another window to fullscreen with `wmctrl`, do nothing on Wayland. Several design decisions in Chapter 5 and Chapter 8 follow from this.

2.5 Tools and Technologies Used

Table 2.3: Tools and technologies used in the project

Area	Technology	Purpose
Kiosk app	C#, .NET 8 LTS, Avalonia 11.2.3	Full-screen desktop UI
QR generation	QRCoder 1.6.0 (PngByteQRCode)	Render the session QR without System.Drawing
PDF display	PDFtoImage 4.1.1 (PDFium + SkiaSharp)	Render report pages inside the app
Backend	Node.js 18+, Express 4, pg	REST API and static web pages
Database	PostgreSQL (Render / Neon)	Visitors, sessions and roster
Web pages	HTML/JS, SheetJS, html5-qrcode	Roster parsing, phone camera scanner
Pipeline	Bash, Python 3.10	Run calibration scripts and meditation-app
Packaging	dotnet publish, dpkg-deb, systemd	Self-contained .deb, auto-start, restart
Hosting	Render (blueprint `render.yaml`)	HTTPS web service + database
Version control	Git, GitHub (branch + pull request per change)	History and review
Preview	Docker, Xvfb, noVNC	View the UI in a browser during development

CHAPTER 3
REQUIREMENTS AND SYSTEM DESIGN

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	On start-up the kiosk opens fullscreen and shows a welcome screen with a QR code.
FR2	Each QR code carries a fresh, hard-to-guess session token created by the backend.
FR3	An administrator can upload an Excel roster and receive one private login link per person.
FR4	A visitor can open their link on a phone, scan the kiosk QR and be identified.
FR5	Visitors can also register themselves on the phone (name, email, age).
FR6	The kiosk detects identification within a few seconds and shows the visitor’s name and photo.
FR7	The kiosk then runs the calibration and meditation pipeline automatically.
FR8	While the pipeline runs, the kiosk shows a live status line.
FR9	When the pipeline finishes, the kiosk shows the new report PDF and a thank-you message.
FR10	A Return button resets the kiosk and creates a new session for the next visitor.
FR11	A manual name entry allows staff to start a session without a phone or server.

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Reliability	Auto-start on boot; restart within seconds after a crash; no state carried between visitors.
Recoverability	Network failures show a clear message and a retry action (“New code”).
Portability	Install on any recent Ubuntu (amd64 or arm64) with nothing pre-installed.
Privacy	Sensitive roster fields (Aadhaar) are never returned by public endpoints; tokens are unguessable.
Usability	At most three taps on the phone; nothing to type at the kiosk.
Maintainability	The pipeline is an editable script; paths are overridable by environment variables.

Table 3.2: Non-functional requirements

Category	Requirement
Performance	Identification visible on the kiosk within one polling interval (2 s).

3.3 System Architecture

Figure 3.1 shows the overall architecture. The system has three zones: the devices of the visitor and the administrator, the cloud backend, and the kiosk machine itself. The kiosk talks to the backend only over HTTPS. The visitor's phone also talks only to the backend; its single link with the kiosk is optical, through the QR code on the screen. On the kiosk, the application starts the hardware team's pipeline as a local process and receives the result as a PDF file.

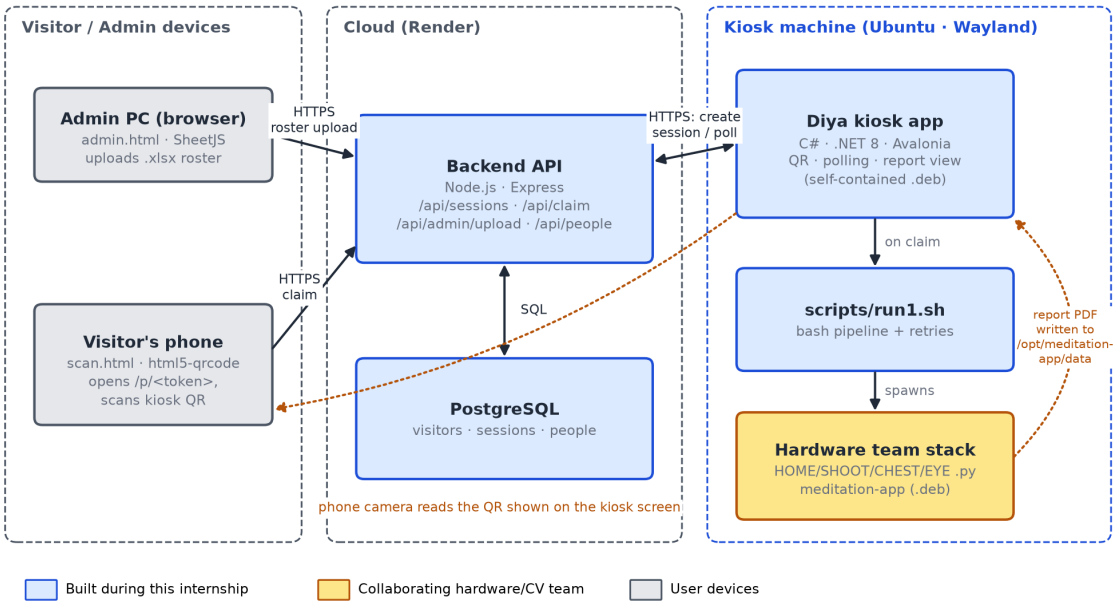


Figure 3.1: Overall system architecture of the Diya kiosk

This structure has one important consequence: **the kiosk and the hardware stack never call each other over an API**. The only contract between them is a script to run and a folder to watch. Each team could therefore change its own internals freely, and the hardware team could test their software without the kiosk app, and vice versa.

3.4 Visitor Flow

The kiosk application behaves as a simple state machine (Figure 3.2). In **Idle** it has a fresh session and shows its QR code. When a phone claims the session it moves to **Claimed** and shows the visitor's details. The pipeline then starts automatically (**Session**), and when the script exits, the newest report is displayed (**Report**). Pressing Return discards everything about the visitor and returns to Idle with a new token.

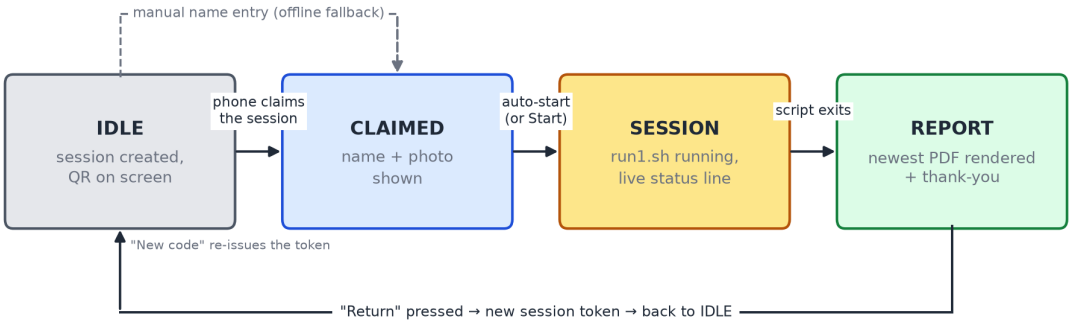


Figure 3.2: Visitor flow as a state machine

3.5 Interaction Sequence

Figure 3.3 shows the complete sequence of messages for one visitor, from the administrator's roster upload to the reset. Solid arrows are HTTP requests to the REST API, dashed arrows are responses, and dotted arrows are interactions that do not use the API at all: the link shared with the visitor, the phone camera reading the screen, and the local pipeline process with its PDF output.

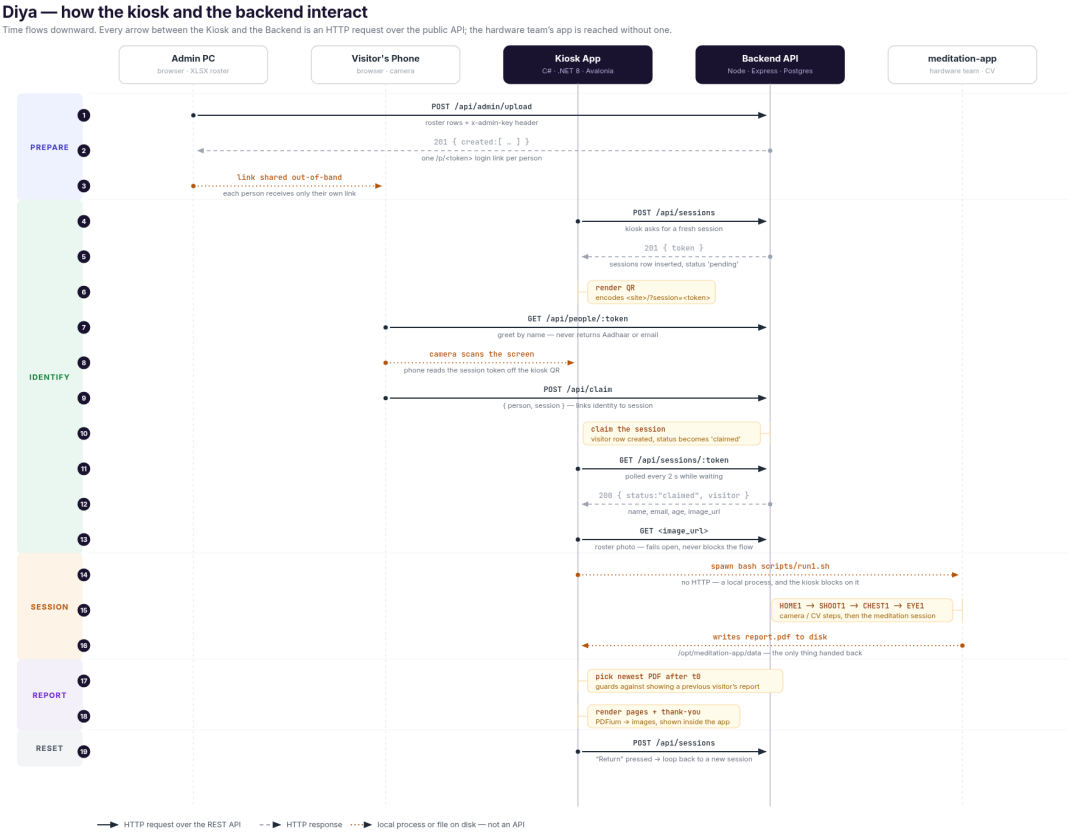


Figure 3.3: Sequence of interactions between the admin, phone, kiosk, backend and meditation-app

3.6 Data Design

The backend uses three PostgreSQL tables (Figure 3.4), created automatically when the server starts. **people** holds the roster uploaded by the administrator; each row has an unguessable `token` that forms the person's private link `/p/<token>`. **sessions** holds one row per QR code shown on the kiosk, with a status of `pending` or `claimed`. **visitors** holds the identity that is linked to a session: when a person claims a session, their roster details are copied into a visitor row, and the session's `visitor_id` is set.

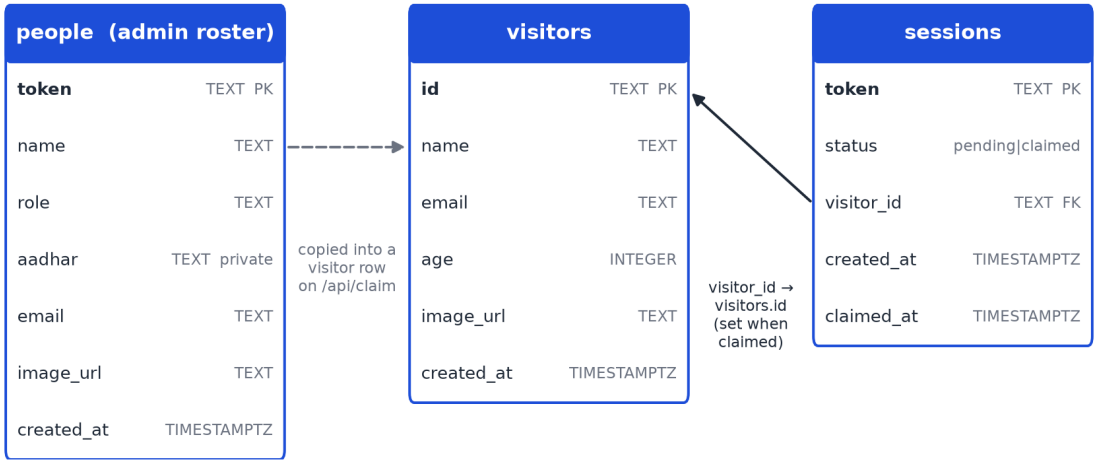


Figure 3.4: Database schema (entity–relationship view)

The Aadhaar number in `people` is the most sensitive field in the system. It is stored only because the roster contains it; it is masked in the admin preview and is **never** returned by any public endpoint.

3.7 API Design

Table 3.3: REST endpoints of the backend

Method	Endpoint	Used by	Purpose
GET	/api/health	Operators	Liveness check, returns {ok:true}
POST	/api/sessions	Kiosk	Create a session; returns {token}
GET	/api/sessions/:token	Kiosk	Poll; returns pending, or claimed with the visitor
POST	/api/visitors	Phone	Self-registration {name, email, age, session?}
GET	/api/visitors/:id	Kiosk	Legacy lookup of a standalone pass
POST	/api/admin/upload	Admin page	Create people from the roster (needs x-admin-key)
GET	/api/people/:token	Phone	Greeting: returns name and role only
POST	/api/claim	Phone	{person, session}: link the person to the session
GET	/admin, /p/:token	Browsers	Serve the admin page and the personal login page

3.8 User Interface Design

The kiosk screen was designed as two panes: the left pane is *who you are* and the right pane is *how you get in*. Before identification the left pane is almost empty, so the QR code is the only thing competing for attention. The wireframes in Figures 3.5 to 3.8 record the reasoning behind each screen, including two flaws that were knowingly accepted for the first release.

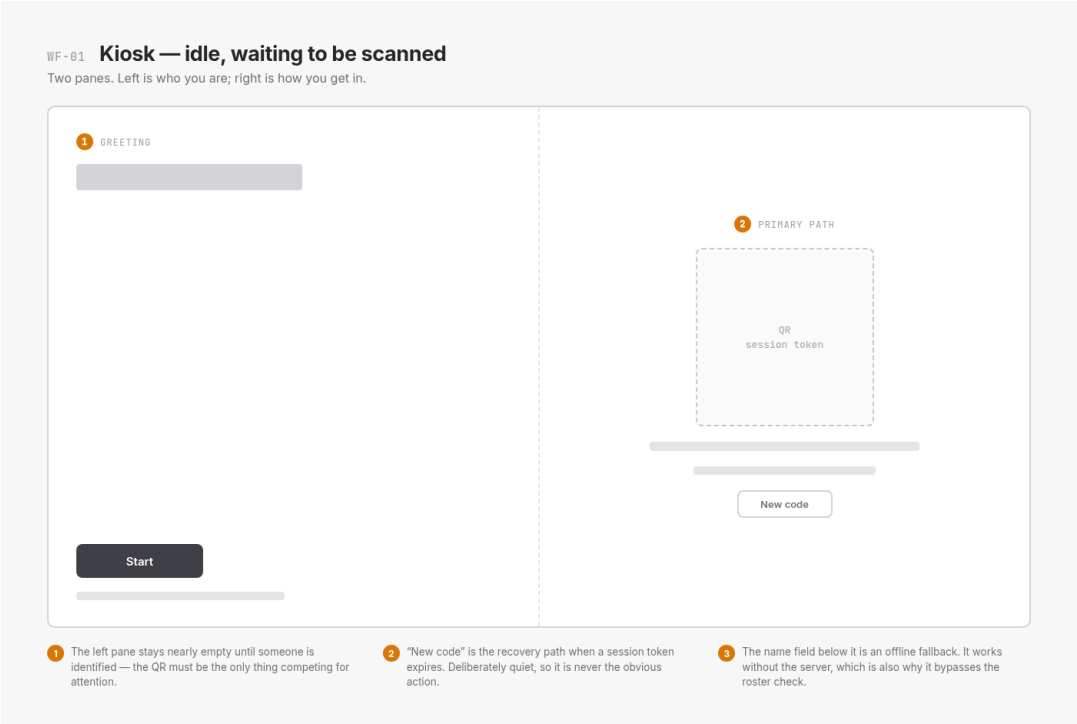


Figure 3.5: Wireframe WF-01: kiosk idle, waiting to be scanned

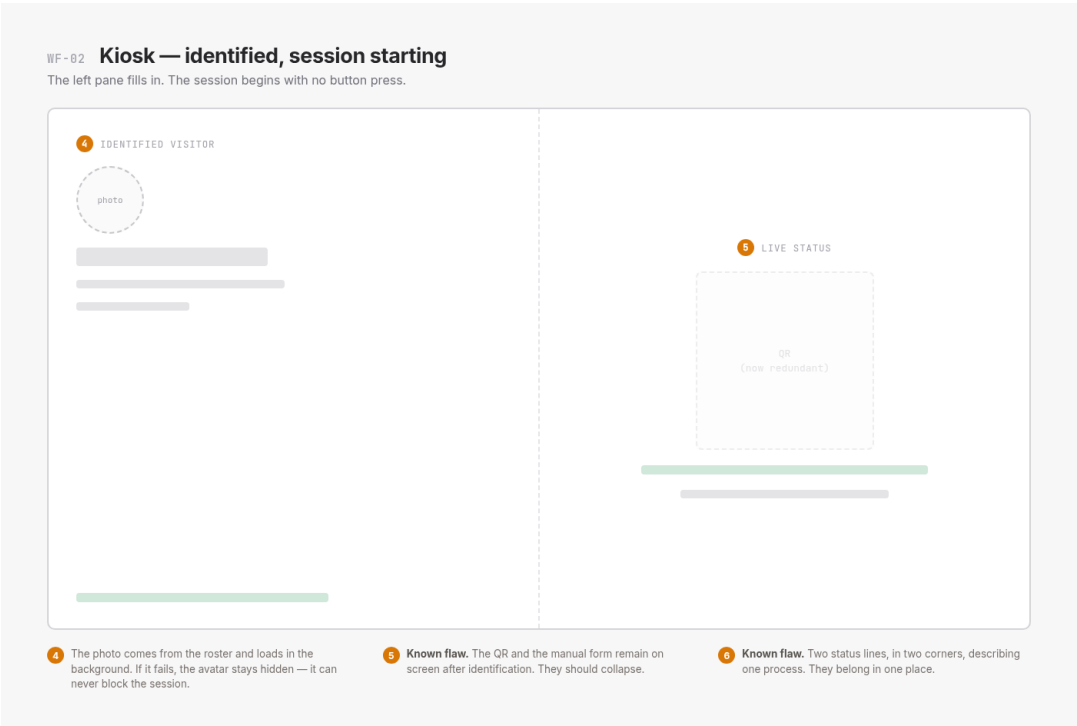


Figure 3.6: Wireframe WF-02: visitor identified, session starting

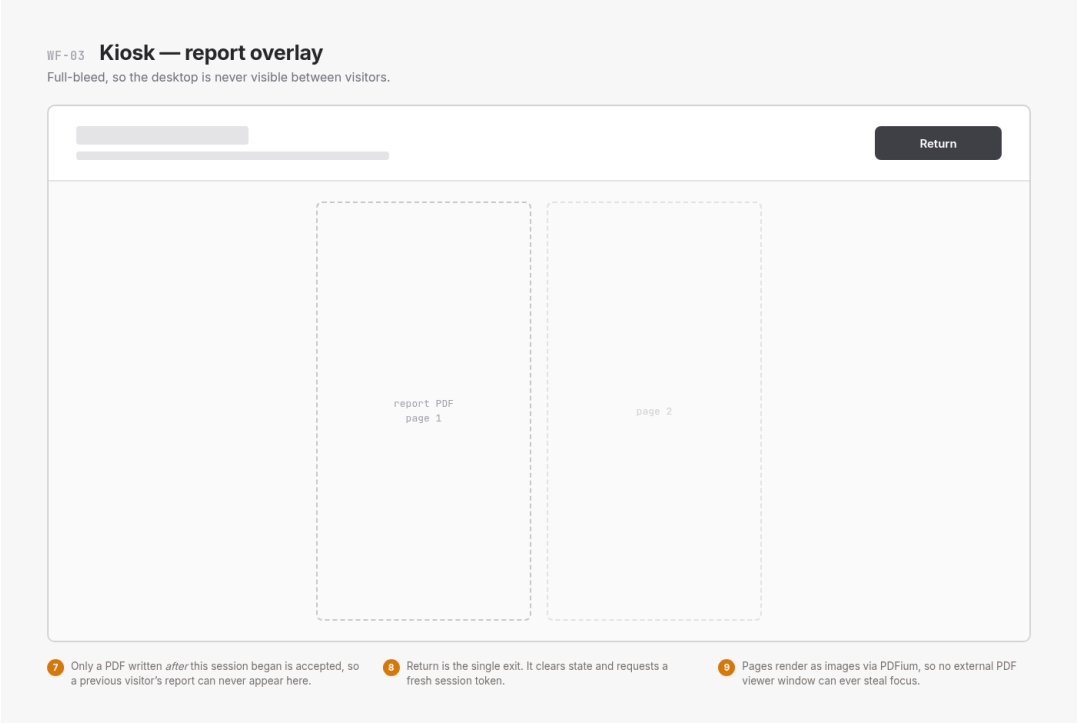


Figure 3.7: Wireframe WF-03: report overlay with the single Return action

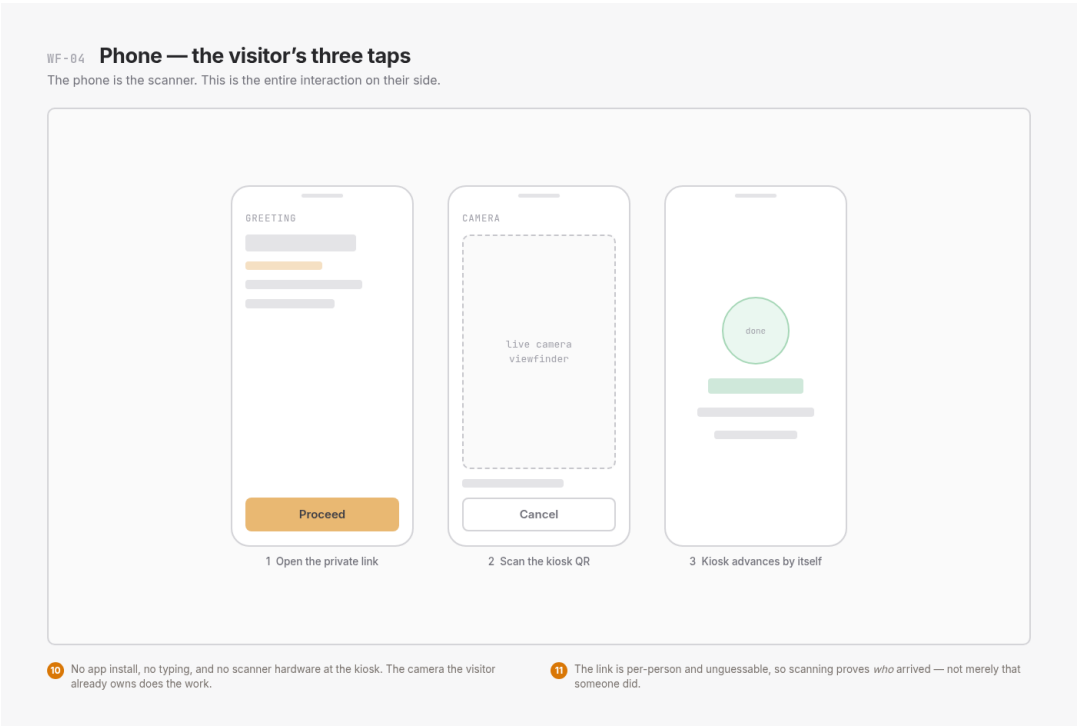


Figure 3.8: Wireframe WF-04: the visitor's three taps on the phone

CHAPTER 4
KIOSK APPARATUS AND HARDWARE SETUP

4.1 Overview of the Apparatus

The Diya kiosk is a physical installation: a seat for the visitor, a display in front of them, and a sensor assembly that carries several cameras. Two motorised axes move the sensors so that one camera lines up with the visitor's chest and another with their eyes. A radar sensor confirms that someone is actually seated before any motion starts. All devices are connected to a single Ubuntu PC that also runs the kiosk application.

The apparatus, its electronics and its control scripts were built by the hardware and computer-vision team. This chapter describes them to the extent needed to understand the kiosk software, because the software had to share the cameras and the screen with them.

4.2 Hardware Components

Table 4.1: Hardware components of the kiosk

Component	Role	Interface
Kiosk PC (Ubuntu, GNOME/Wayland)	Runs the kiosk app, CV scripts and meditation-app	—
Display	Kiosk UI, meditation video, report	HDMI
Logitech C920 webcam	Face and gaze capture	USB (V4L2)
Intel RealSense D435	Depth capture	USB (V4L2 / librealsense)
Arducam 8MP	Front view and posture	USB (V4L2)
USB thermal camera	Thermal capture	USB (V4L2)
Arduino controller	Drives the motors, reads switches and sensors	USB serial, 115200 baud, ASCII line protocol
Motors M1 and M2	M1 moves the chest axis, M2 the eye axis; homing and actuator deployment	Via Arduino
Limit switches and proximity sensors	Home position, collision detection, early warning	Via Arduino (paired NO+NC home switches)
LD2410 mmWave radar	Detects a seated person in a 650–1050 mm band	USB serial, 256000 baud

4.3 Hardware Block Diagram

Figure 4.1 shows how the components connect. The cameras are ordinary USB video devices; the motors and safety switches are reached only through the Arduino, which accepts short text commands such as `home1`, `shoot m1` or a streamed alignment error `E:<value>` and reports events such as `DONE:HOME1` or `COLLISION:ACTIVE`.

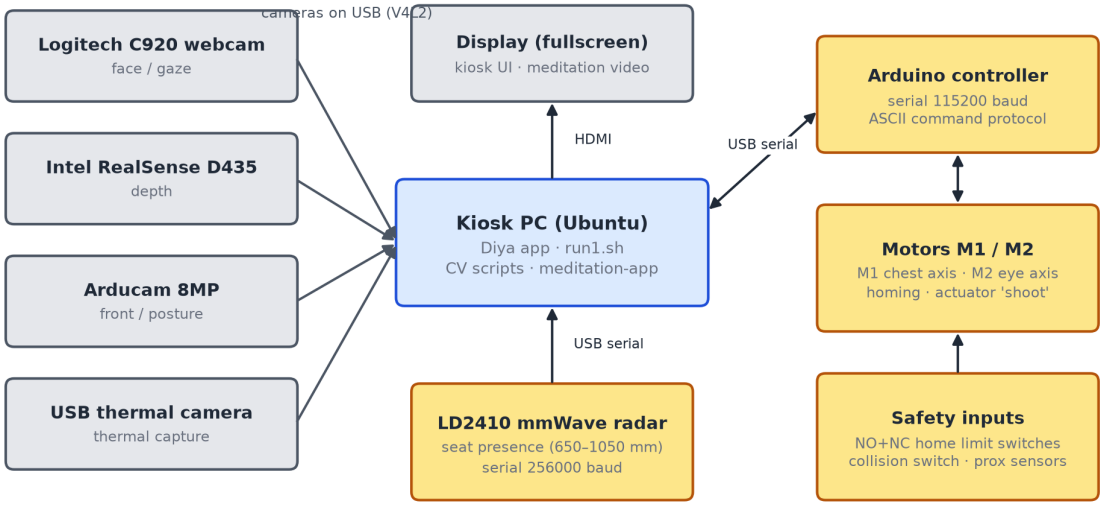


Figure 4.1: Hardware block diagram of the kiosk

A notable safety feature of the hardware team's design is the wiring of each home limit switch as a *normally-open plus normally-closed pair*. A self-test can then tell apart a switch that is open, a cut wire and a short circuit, and the firmware reports a fault before any motor is moved.

4.4 Photographs of the Apparatus

Figures 4.2 to 4.7 show the physical kiosk and its main assemblies.

PLACE PHOTOGRAPH HERE

Insert photograph: the complete kiosk as a visitor sees it (seat, display and sensor assembly).

Figure 4.2: Front view of the Diya meditation kiosk

PLACE PHOTOGRAPH HERE

Insert photograph: side view showing the seating position relative to the cameras.

Figure 4.3: Side view of the kiosk showing the seat and sensor arm

PLACE PHOTOGRAPH HERE

Insert photograph: close-up of the camera mounts.

Figure 4.4: Camera assembly (C920, RealSense D435, Arducam, thermal camera)

PLACE PHOTOGRAPH HERE

Insert photograph: motors M1/M2, the axes they move and the limit switches.

Figure 4.5: Motorised chest and eye axes with the actuator mechanism

PLACE PHOTOGRAPH HERE

Insert photograph: controller board, motor drivers and switch wiring.

Figure 4.6: Arduino controller and wiring

PLACE PHOTOGRAPH HERE

Insert photograph: radar module and where it is mounted relative to the seat.

Figure 4.7: LD2410 radar presence sensor and its mounting

4.5 Software–Hardware Boundary

The kiosk application does not drive any hardware itself. When a visitor is identified, it releases control to a bash script, `run1.sh`, which runs the hardware team's stages in order and then starts `meditation-app`. Inside `meditation-app` a one-minute guided video plays fullscreen, the camera stages and a roughly five-minute multi-camera acquisition run, generated clips are played back, and a PDF report is built. The report is written to `/opt/meditation-app/data`. When the script exits, the kiosk picks up that PDF and shows it. This boundary is drawn as the dotted arrows in Figure 3.1.

CHAPTER 5
IMPLEMENTATION

5.1 Structure of the Kiosk Application

The kiosk application is deliberately small: one window, one main screen and a few services. Table 5.1 lists its parts.

Table 5.1: Modules of the kiosk application

File / module	Responsibility
Program.cs, App.axaml(.cs)	Entry point, Fluent light theme, Inter font; ShutdownMode set to OnExplicitShutdown so the process outlives its window
Views/MainWindow	Fullscreen shell that hosts the main screen; re-asserts fullscreen after opening
Views/HomeView	The whole visitor experience: session QR, polling, visitor details and photo, manual fallback, pipeline status, report overlay, Return
Models/VisitorApiClient	HTTP client for the backend (create session, poll, fetch photo); base URL from DIYA_API_BASE
Models/VisitorData, VisitorQr	Visitor record; parser for the legacy offline QR pass
Services/PipelineRunner	Runs scripts/run1.sh with bash, streams its output, waits for exit
Services/ReportRenderer	Finds the newest PDF written since the session began and renders its pages to bitmaps
scripts/run1.sh, fullscreen-fixer.sh	Editable pipeline with retry logic; helper that fullscreens the meditation windows on X11
deploy/*	Debian package builder, kiosk setup script, systemd unit, autostart entry

5.2 Fullscreen Kiosk Shell on Wayland

Making the window reliably fullscreen on GNOME/Wayland took several iterations in the first week. The lessons, all of which are now built into the application and its package, were:

1. **The window must remain resizable.** Marking it non-resizable sets fixed size hints, and the compositor then refuses to make it fullscreen.
2. **Apply fullscreen in code, not in XAML.** Setting the state in markup does not survive on Wayland, so it is applied when the window opens.

3. **Re-assert fullscreen after opening.** The compositor first maps the window in its normal state; a timer re-applies fullscreen six times at 250 ms intervals and then stops, so it never fights the user afterwards.
4. **Drop the ICU dependency** (`InvariantGlobalization`), so the package installs on any Ubuntu release without a matching libicu version.
5. **Publish self-contained, single-file**, so the .NET runtime is bundled and nothing has to be installed first.
6. **Delay auto-start by a few seconds**, because an application launched before the Wayland session is ready fails silently.

Early versions also vetoed the window's close event and offered only a secret key combination to exit. These were later removed at the lab's request so that staff could minimise and close the application during development; crash recovery is still handled by the system service.

5.3 Session QR, Polling and the Home Screen

When the home screen appears, it asks the backend for a new session. Because the free hosting tier puts the server to sleep after about fifteen minutes of inactivity, the first request can take 30–50 seconds, so the status line reads *"Waking up the server…"* and the HTTP client allows up to 60 seconds. If the request still fails, the screen says so and offers **New code** to retry.

Once a token arrives, the app encodes `<site>/?session=<token>` as a QR image using QRCoder and starts a 2-second timer that polls `GET /api/sessions/:token`. As soon as the status becomes `claimed`, polling stops and the left pane fills in with the visitor's name, email, age and roster photo. The photo is downloaded in the background and *fails open*: if it cannot be fetched, the avatar simply stays hidden and the session continues. Below the QR, a small form lets staff type a name and start a session without a phone or a server, which is also how the system is tested offline.

5.4 Backend Service

The backend is a single Express application of about 340 lines with a PostgreSQL connection pool. It creates its schema on start-up, so a new deployment needs no migration step. Its main design points are:

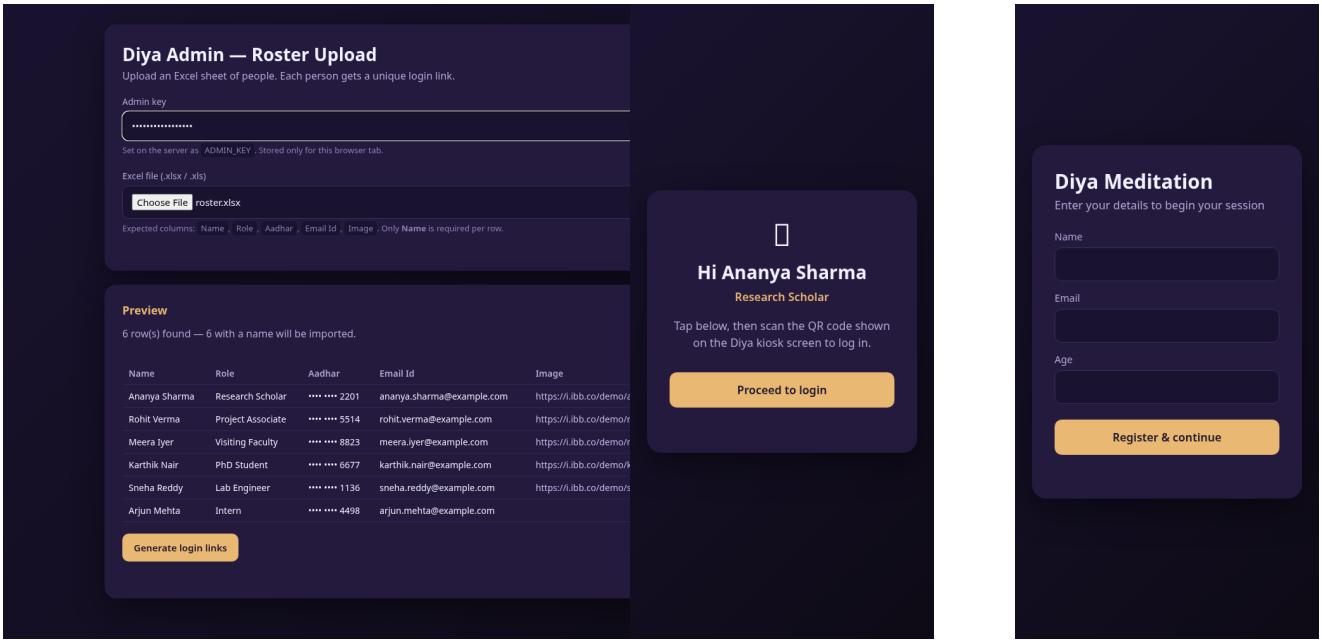
- **Unguessable tokens.** Session tokens are 9 random bytes (about 12 URL-safe characters) and personal link tokens are 16 random bytes (about 22 characters), generated with Node's cryptographic random generator.

- **Claim only once.** A session is linked to a visitor with an update that only matches rows whose status is still `pending`, so a token cannot be claimed twice.
- **Minimal disclosure.** `GET /api/people/:token` returns only the name and role needed to greet the person; Aadhaar and email never leave the server through public endpoints.
- **Admin protection.** Roster upload requires the `ADMIN_KEY` secret in a request header. If the key is not configured, the admin feature is disabled entirely.
- **Same origin.** The API also serves the registration, admin and login pages, so one HTTPS service is enough. HTTPS is mandatory because phone browsers only allow camera access on secure pages.

5.5 Web Pages for Admin and Phone

admin.html lets an administrator choose an Excel file. It is parsed entirely in the browser with SheetJS; column headers are matched loosely (case-insensitive substrings), so “Image”, “Image URL” or “Image Link (GDrive …)” are all recognised. A preview with masked Aadhaar numbers is shown before anything is sent. After upload, the page lists one personal link per person, with *Copy all* and *Download as CSV*.

scan.html is served at `/p/<token>`. It greets the person by name and, on *Proceed to login*, opens the phone camera using the html5-qrcode library, reads the session token from the kiosk QR, and calls `/api/claim`. **index.html** offers self-registration for visitors who are not on a roster.



(a) Admin roster upload (b) Personal login link (c) Self-registration

Figure 5.1: Web pages of the system (sample data)

5.6 Pipeline Integration and Report Display

When a visitor is identified, `PipelineRunner` starts `bash scripts/run1.sh` with `UseShellExecute=false` and all output redirected, so no terminal window appears. Each line the script prints is shown, shortened to 140 characters, as the live status on screen; this is how the visitor sees messages such as *"Calibrating cameras…"*. There is intentionally **no timeout** in the application: the script owns its retry logic, and the meditation itself takes several minutes.

The script first checks that the calibration scripts and `meditation-app` are present, then runs the stages with the retry rules shown in Figure 5.1: the homing and actuator stages retry themselves, while a failure in chest or eye alignment restarts the whole sequence from homing, because the rig must be re-homed before it can be aligned again.

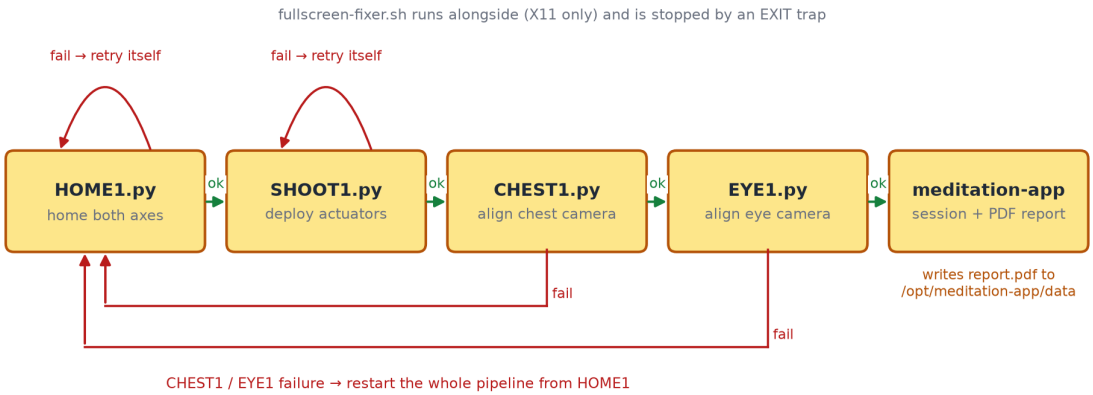


Figure 5.2: Retry logic of the pipeline script run1.sh

When the script exits, `ReportRenderer` looks in `/opt/meditation-app/data` for the newest PDF whose modification time is **after the session started**. This simple rule guarantees that a previous visitor's report is never shown to the next person. Each page is rendered to an image with PDFium and shown inside the app, so no external PDF viewer can open a window and take focus. A personalised thank-you line and a single **Return** button complete the screen.

5.7 Packaging and Deployment

`deploy/build-deb.sh` publishes the application for `amd64` or `arm64` as a self-contained single file, adds the pipeline scripts, a desktop entry and a systemd user service, and assembles a Debian package. The package depends only on a few common libraries (`libx11-6`, `libice6`, `libsm6`, `libfontconfig1`) and `python3`. The installation creates `/opt/meditation-app/data` and gives the kiosk user write access to it. If `dpkg-deb` is unavailable, the script falls back to building the archive with `ar` and `tar`.

On the kiosk, a systemd user service starts the app with the graphical session and restarts it two seconds after any exit (`Restart=always`). `setup-kiosk.sh` enables automatic login and lingering, and disables GNOME shortcuts that could let a visitor escape. The backend is deployed on Render from the blueprint `render.yaml`, which creates the web service and a PostgreSQL database and connects them. Figure 5.2 summarises the deployment, and Table 5.2 lists the settings.

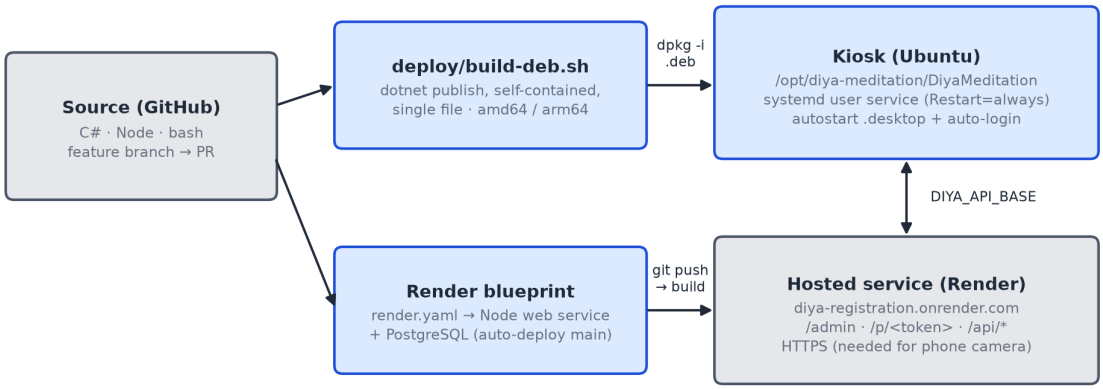


Figure 5.3: Build and deployment of the kiosk and the backend

Table 5.2: Configuration through environment variables

Variable	Where	Meaning
DIYA_API_BASE	Kiosk	Backend URL (default: the Render service)
DIYA_PIPELINE_SCRIPT	Kiosk	Script to run on login (default: bundled run1.sh)
DIYA_BASH	Kiosk	Shell used to run the script
DIYA_REPORT_DIR	Kiosk	Folder to read the report from (default /opt/meditation-app/data)
DATABASE_URL	Server	PostgreSQL connection string
ADMIN_KEY	Server	Secret that enables and protects roster upload
PGSSL, PORT	Server	Disable SSL for local databases; listening port

5.8 Screens of the Final Application

Figures 5.3 to 5.6 are genuine frames of the application, rendered through Avalonia's headless platform with the same layout, fonts and QR encoder as on the kiosk. The visitor shown is fictional, and the report is a representative sample, because the real report comes from the hardware team's software.

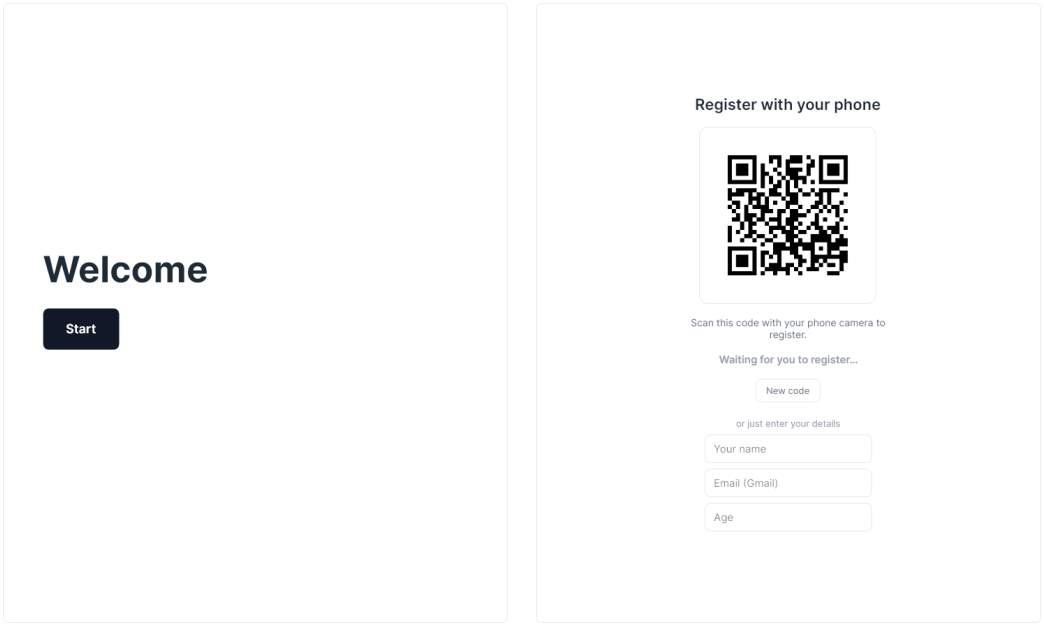


Figure 5.4: Kiosk idle: session created and QR code displayed

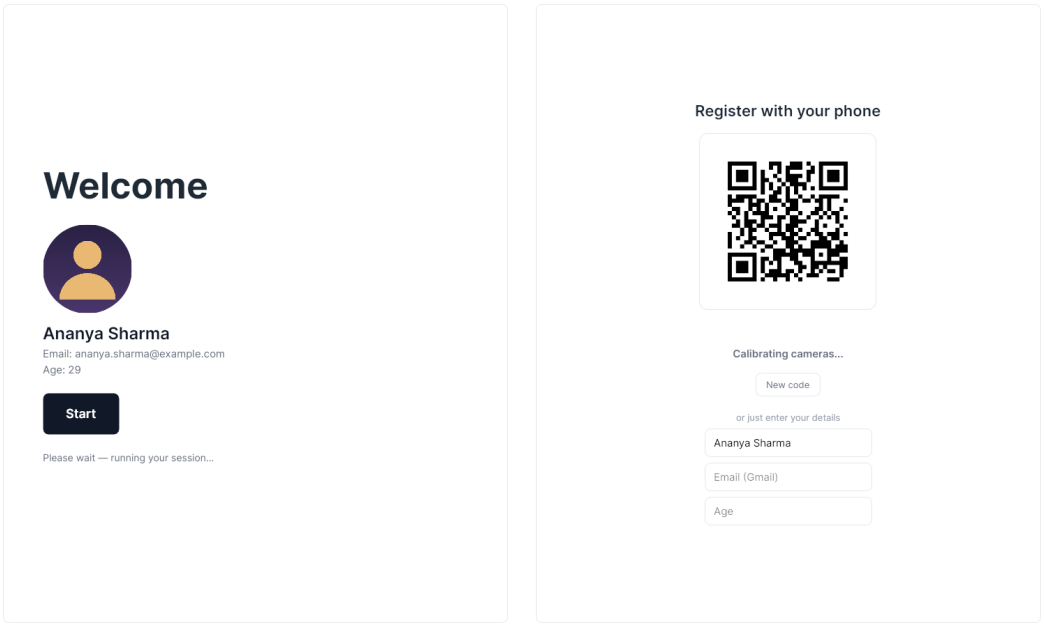


Figure 5.5: Kiosk after the phone has claimed the session

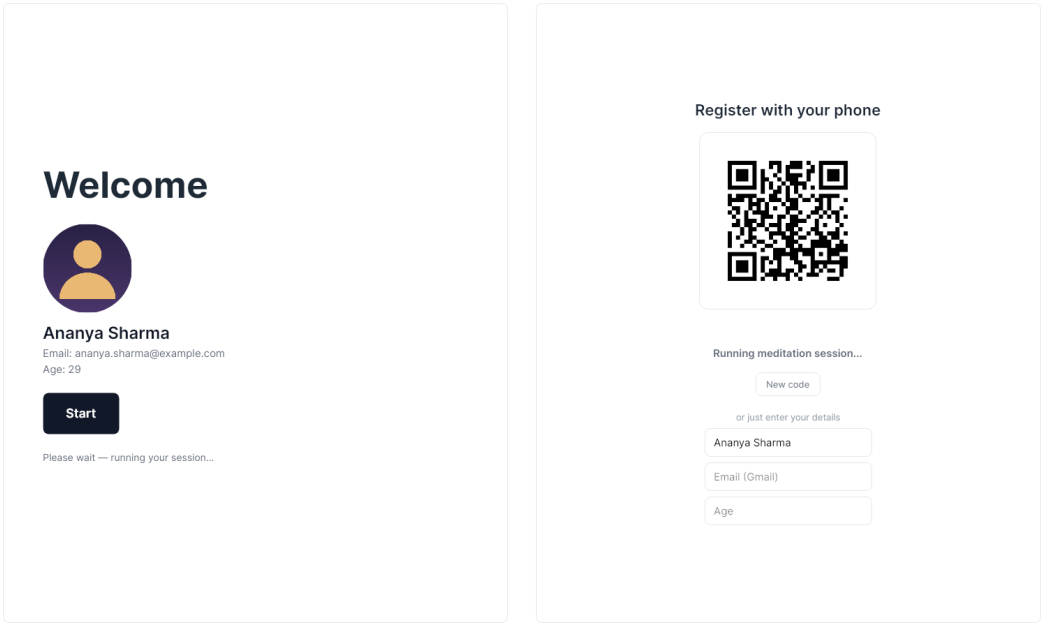


Figure 5.6: Pipeline running, with the live status line

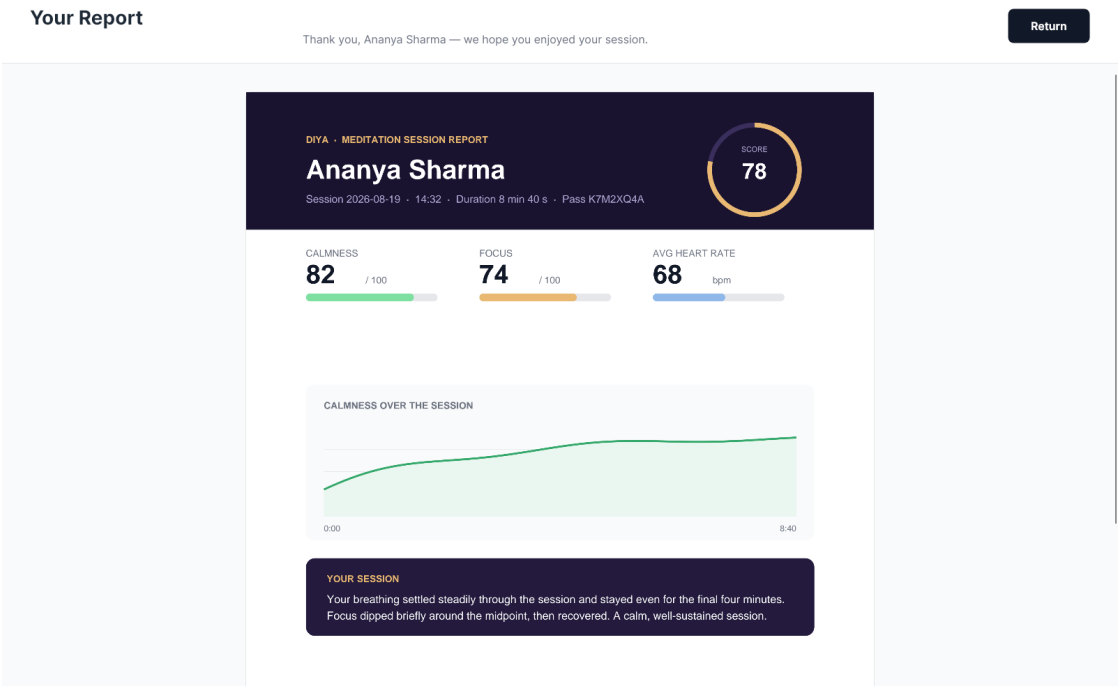


Figure 5.7: Report displayed in the app with the thank-you message and Return

CHAPTER 6
EVOLUTION OF THE DESIGN

6.1 Iterative Development

The project was built in small steps. Every change was made on its own branch and merged through a pull request, and a new Debian package was published whenever the kiosk needed to be updated. Over the eight weeks, 28 pull requests were merged and seven versions were released. Figure 6.1 and Table 6.1 show how the visitor identification, in particular, changed as the constraints became clearer.

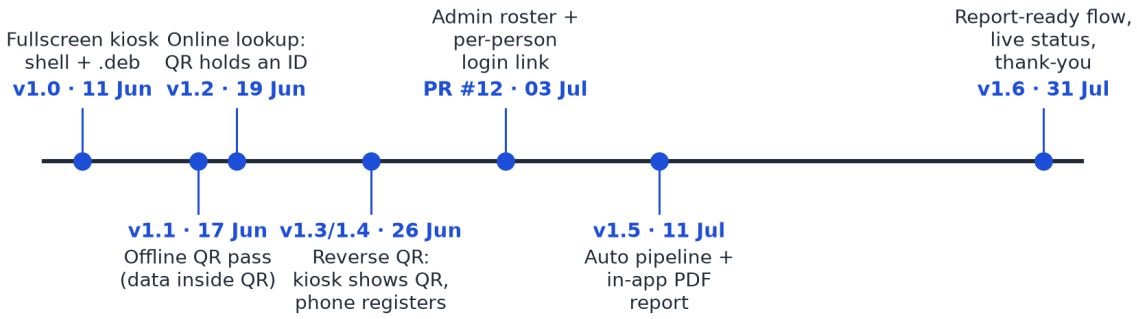


Figure 6.1: Timeline of the main versions

Table 6.1: Version history

Version	Date	Change
1.0.x	11–13 Jun	Fullscreen kiosk shell, Wayland fixes, .deb for amd64 and arm64
1.1.0	17 Jun	Offline QR pass: visitor details encoded inside the QR
1.2.0	19 Jun	Online lookup: QR carries only an ID; hosted API and database
1.3.0	26 Jun	Reverse flow: kiosk shows the QR, phone registers, kiosk advances itself
1.4.0	26 Jun – 9 Jul	Calibration launch, manual fields, admin roster and phone login, visitor photo
1.5.0	11 Jul	Login runs the pipeline automatically and shows the PDF report in the app
1.6.0	31 Jul	Report-ready flow, live status, thank-you message, no-camera test kit

6.2 From Data in the QR to a Session Token

The first version put the visitor's name, email and age directly into the QR code as Base64-encoded JSON with a `DIYA1:` prefix, so the kiosk could work fully offline. It worked, but anyone could read the data in the code, and a pass could not be revoked. Version 1.2 moved the data to a hosted database and left only a short ID in the code.

6.3 Inverting the QR Flow

The decisive change came from a practical constraint: **the kiosk had no QR scanner**, and its cameras were reserved for the meditation pipeline. Rather than add hardware, version 1.3 reversed the roles. The kiosk now *shows* a QR code holding a short-lived session token, and the visitor's phone *reads* it. This needs no app installation and nothing to type at the kiosk. With the admin roster added in early July, each person received a private link, so the scan also proves *who* is standing at the kiosk, not merely that somebody scanned. A constraint produced a simpler design.

6.4 Alternatives That Were Explored

Two alternative directions were built on separate branches at the end of June and kept for comparison rather than merged:

- **Offline people list:** the kiosk loads a fixed CSV/XLSX file, and visitors are identified by scanning a printed pass with a USB scanner or typing their name. A small tool generated the printable passes. This suits sites without internet.
- **Full in-app flow:** separate calibration, meditation and report screens inside the kiosk app, with mock sensor data and a report generated by the app itself. This was set aside once it was clear that the hardware team's `meditation-app` would run the session and produce the report.

6.5 Window Hand-off Experiment and Rollback

In late July the lab wanted the kiosk to step aside while `meditation-app` was on screen and come back afterwards. Version 1.5 was extended so the kiosk minimised itself during the session. On Wayland, however, an application is not allowed to restore or raise itself, so the kiosk appeared to close after login and could not return. After two attempts to fix it, the change and the follow-ups built on it were **rolled back** to the last stable baseline (pull request #24), and the useful parts, the live status line, the thank-you message and the test kit, were re-implemented cleanly as version 1.6.0. The lesson is discussed in Chapter 8.

CHAPTER 7
TESTING AND RESULTS

7.1 Testing Approach

Testing was done at three levels. Automated checks exercised the backend session flow and the kiosk's HTTP client end to end. Each screen was constructed and rendered through Avalonia's headless platform, which also produced the screenshots in Chapter 5. Finally, every package was installed and run on the real kiosk by the lab, and the full flow was demonstrated with a real phone.

7.2 Test Cases

Table 7.1: Summary of test cases

#	Scenario	Expected result	Outcome
T1	Create session → register with token → poll	Poll returns claimed with the visitor	Passed (automated)
T2	Kiosk client: create, claim, poll, unknown token	Pending → Claimed; unknown token is NotFound	Passed (automated)
T3	Render every screen headlessly	Screens build without errors	Passed
T4	Install .deb on different Ubuntu releases	Installs without dependency errors	Passed after removing the libicu dependency (v1.0.x)
T5	Start on GNOME/Wayland	Window covers the whole screen	Passed after the fullscreen fixes
T6	First request to a sleeping server	Kiosk waits, then shows the QR	Passed (60 s timeout, "Waking up the server…")
T7	Network unavailable	Clear error and retry with New code	Passed
T8	Roster photo URL broken	Session continues without photo	Passed (fails open)
T9	Full flow with mock pipeline (test kit)	Status updates, sample report, thank-you	Passed
T10	Phone scans the kiosk QR in a live run	Kiosk advances by itself	Passed (recorded demo)
T11	Kiosk minimises itself during the session	Kiosk returns afterwards	**Failed on Wayland**; change rolled back

Table 7.1: Summary of test cases

#	Scenario	Expected result	Outcome
T12	Kiosk clock set in the past	HTTPS connection succeeds	Fails (certificate “not yet valid”); fixed by enabling NTP

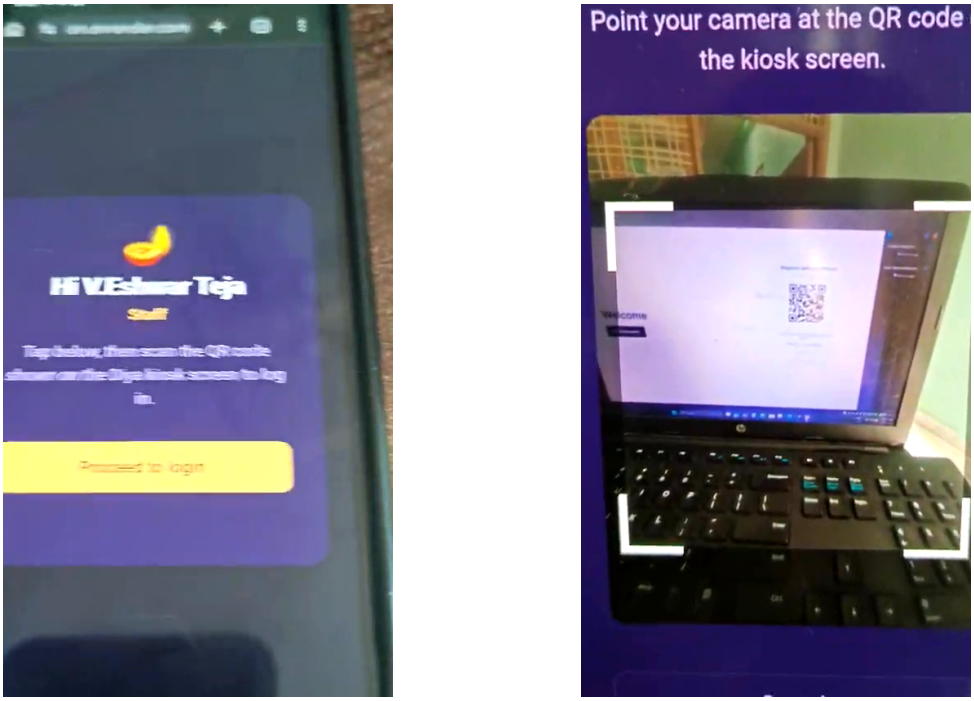
7.3 Testing Without Cameras

Because the real pipeline needs the cameras, the motors and `meditation-app`, I wrote a small test kit that replaces it. Two environment variables point the kiosk at a mock script and a temporary report folder; the mock prints status lines, waits about eight seconds and drops a sample PDF. No test code is included in the application or the package, so the kiosk that is tested is exactly the kiosk that ships.

```
export DIYA_PIPELINE_SCRIPT="$(pwd)/testkit/run1.mock.sh"
export DIYA_REPORT_DIR=/tmp/diya-reports
mkdir -p /tmp/diya-reports
diya-meditation      # type a name and press Start
```

7.4 Field Demonstration

Figure 7.1 shows frames from a recorded run with the deployed backend. The visitor opens their personal link on a phone, which greets them by name; they point the camera at the QR on the kiosk screen; and the kiosk advances on its own and shows their details. The recording was made with an earlier build on a development laptop, so some labels (for example *Start Calibration*) differ from the final version.



(a) Personal link greets the visitor (b) Phone scanning the kiosk QR

Figure 7.1: Live demonstration: the visitor's phone

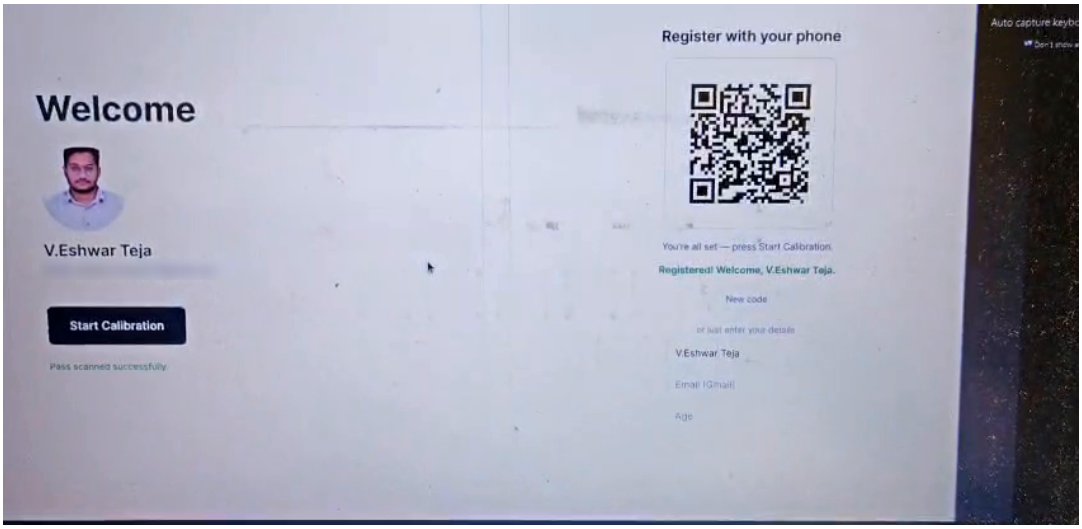


Figure 7.2: Live demonstration: the kiosk after the phone has claimed the session (email address blurred)

7.5 Results

- The complete visitor journey (identify, claim, pipeline, report, reset) works end to end, both with the real backend and phone and with the offline test kit.
- Identification needs no scanner, keyboard or app installation; the visitor's side is three taps.

- The kiosk installs from a single package on amd64 and arm64 and recovers from crashes through systemd.
- A previous visitor’s report can never be shown to the next person, and no terminal or PDF-viewer window appears from the kiosk’s side.
- Seven versions were delivered in eight weeks, each with updated setup documentation.

CHAPTER 8
CHALLENGES AND LEARNINGS

8.1 Challenges Faced

Table 8.1: Main challenges and how they were addressed

Challenge	Resolution
Fullscreen did not stick on GNOME/Wayland	Keep the window resizable, apply fullscreen in code and re-assert it briefly after opening
Package failed on some Ubuntu versions	Bundle the runtime and drop the ICU dependency
No QR scanner at the kiosk	Invert the flow: the kiosk shows the QR, the phone scans it
Integrating another team’s software without an API	Contract of a script plus a report folder; pick only PDFs newer than the session start
Free server sleeps when idle	Long client timeout and a clear “waking up” message
Phone camera blocked on plain HTTP	Serve all pages from the HTTPS service
Roster images given as share-page links or HEIC	Document that a direct JPG/PNG URL is required; loose header matching in the admin page
Kiosk could not restore itself after minimising	Roll back to the stable baseline; avoid self-managing windows on Wayland
Stray windows from the meditation app	Fullscreen helper on X11; documented a tracing method for the hardware machine

8.2 Learnings

- **Constraints can simplify a design.** The missing scanner led to a flow that is easier for visitors and needs less hardware.
- **Integrate through the narrowest possible contract.** A script and a folder were enough, and they let two teams move independently.
- **Know the platform’s rules before designing around them.** Wayland’s window policy is intentional; fighting it cost two releases.
- **Rolling back is a valid engineering decision.** Returning to a known-good baseline and re-applying only the useful parts was faster than patching a broken design.
- **Write things down.** Keeping the setup guide updated with every merge meant the lab could install and troubleshoot each build without me.

- **Privacy is part of the design.** Masking, minimal endpoints and unguessable tokens had to be planned in from the start because the roster contains ID numbers and photos.

CHAPTER 9
CONCLUSION AND FUTURE WORK

9.1 Conclusion

During this internship I designed and built the software side of the Diya meditation kiosk: a fullscreen Avalonia application for Ubuntu, a Node.js and PostgreSQL backend, phone-facing web pages, an integration layer for the hardware team's pipeline, and a self-contained installer with automatic start and crash recovery. The kiosk identifies a visitor with nothing more than the visitor's own phone, runs the calibration and meditation pipeline, shows the resulting report and resets for the next person.

Beyond the features, the project taught me how to design for a machine that runs unattended, how to work across team boundaries through small and stable interfaces, and how to make and undo decisions quickly when the platform does not behave as expected.

9.2 Future Work

1. **Consent and privacy screen** before any camera starts, explaining what is captured and stored.
2. **Inactivity reset**, so the kiosk returns to the welcome screen if a visitor leaves mid-session.
3. **Timeouts and a visible error screen** for the pipeline, so a stage that never succeeds cannot hang the kiosk.
4. **Real calibration stages:** replace the placeholder scripts called by `run1.sh` with the hardware team's motor-control and radar stages, adding an overall timeout to the alignment loops.
5. **Face recognition** as an alternative way to identify enrolled visitors, building on the face-recognition team's identify service.
6. **Report delivery** to the visitor by email or message.
7. **Continuous integration** that builds the packages and publishes them as releases, with a single source for the version number.
8. **Persistent database** (for example Neon or Supabase) instead of the free tier, which deletes data after 30 days.

REFERENCES

[1] Avalonia UI. *Avalonia Documentation*. https://docs.avaloniaui.net/

[2] Microsoft. *.NET 8 documentation: single-file deployment and self-contained apps*. https://learn.microsoft.com/dotnet/core/deploying/

[3] Microsoft. *Globalization invariant mode in .NET*. https://learn.microsoft.com/dotnet/core/runtime-config/globalization

[4] QRCoder. *QRCoder: a pure C# QR code implementation*. https://github.com/codebude/QRCoder

[5] PDFtoImage. *PDFtoImage: render PDF pages with PDFium*. https://github.com/sungaila/PDFtoImage

[6] ISO/IEC 18004:2015. *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*.

[7] OpenJS Foundation. *Express 4.x API reference*. https://expressjs.com/

[8] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. https://www.postgresql.org/docs/

[9] node-postgres. *pg: PostgreSQL client for Node.js*. https://node-postgres.com/

[10] Render. *Blueprint specification (render.yaml)*. https://render.com/docs/blueprint-spec

[11] SheetJS. *SheetJS Community Edition documentation*. https://docs.sheetjs.com/

[12] mebjas. *html5-qrcode*. https://github.com/mebjas/html5-qrcode

[13] MDN Web Docs. *MediaDevices.getUserMedia() — secure context requirement*. https://developer.mozilla.org/docs/Web/API/MediaDevices/getUserMedia

[14] freedesktop.org. *systemd.service — service unit configuration*. https://www.freedesktop.org/software/systemd/man/systemd.service.html

[15] Wayland. *The Wayland Protocol*. https://wayland.freedesktop.org/docs/html/

[16] Debian. *Debian Policy Manual: binary packages*. https://www.debian.org/doc/debian-policy/

[17] Intel. *Intel RealSense D400 series product documentation*. https://www.intelrealsense.com/

[18] Hi-Link. *HLK-LD2410 24 GHz human presence sensor: user manual and serial protocol*.

APPENDIX A
WEEKLY WORK LOG

Figure A.1 shows the work plan as carried out, and Table A.1 lists the main work of each week. Dates after the first week are taken from the project's version-control history.

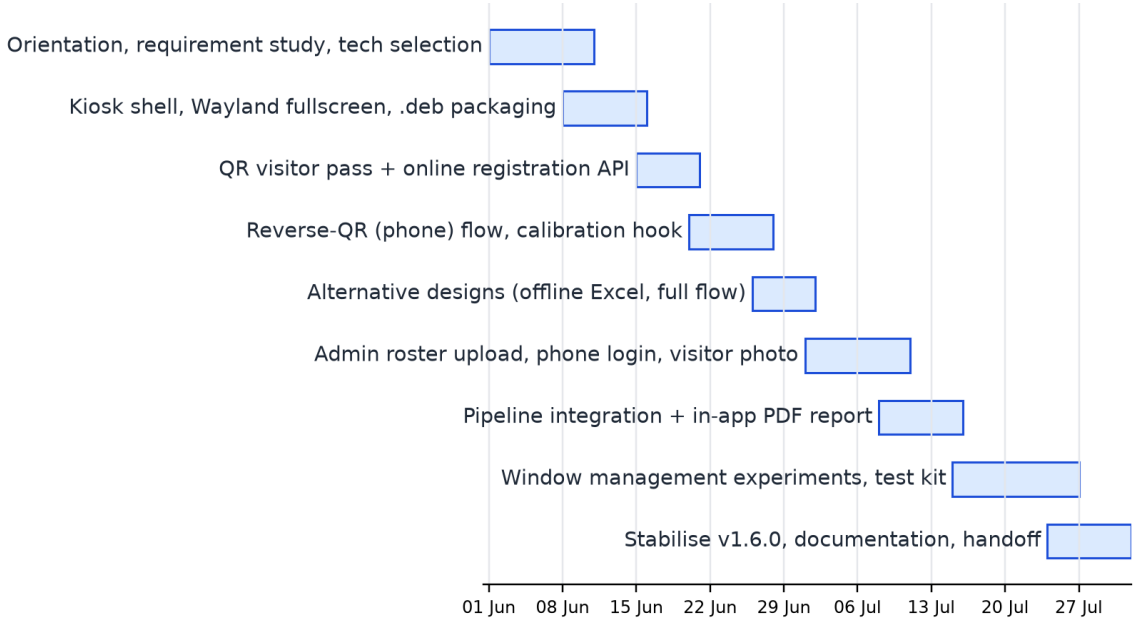


Figure A.1: Gantt chart of the internship (1 June – 31 July 2026)

Table A.1: Weekly work log

Week	Dates	Work done
1	1–7 Jun	Orientation at the lab; study of the kiosk concept; requirements; choice of C#/.NET 8 and Avalonia
2	8–14 Jun	First kiosk app; minimal two-panel home screen; fullscreen fixes for GNOME/Wayland; .deb builder for amd64 and arm64; setup guide
3	15–21 Jun	Offline install instructions; QR visitor pass (v1.1.0); hosted API and database with lookup by ID (v1.2.0)
4	22–28 Jun	Reverse QR flow with phone registration (v1.3.0); calibration launch and manual fields (v1.4.0)
5	29 Jun – 5 Jul	Alternative branches (offline people list, full in-app flow); project overview and FAQ; admin roster upload and reverse-scan login
6	6–12 Jul	Visitor photo on the kiosk; header-matching fix; setup docs for roster and reinstall; login runs the pipeline and shows the PDF (v1.5.0)
7	13–19 Jul	Window behaviour changes requested by the lab (close and minimise allowed); testing on the kiosk

Table A.1: Weekly work log

Week	Dates	Work done
8	20–26 Jul	Codebase overview document; window hand-off experiment and no-camera test kit; Wayland problems found
9	27–31 Jul	Rollback to the stable baseline; report-ready flow, live status and thank-you (v1.6.0); test kit; handoff document

APPENDIX B
INSTALLATION AND OPERATION COMMANDS

Install or update the kiosk package (stop the running app first when updating):

```
systemctl --user stop diya-meditation 2>/dev/null; pkill -f DiyaMeditation
sudo dpkg -i ./diya-meditation_1.6.0_amd64.deb
sudo apt -f install          # only if a dependency is reported missing
diya-meditation              # or: systemctl --user start diya-meditation
```

Point the kiosk at the backend and check that it is reachable:

```
DIYA_API_BASE=https://diya-registration.onrender.com diya-meditation
curl https://diya-registration.onrender.com/api/health     # {"ok":true}
```

Build the package from source:

```
cd DiyaMeditation
./deploy/build-deb.sh 1.6.0 amd64     # or arm64
```

Run the backend locally:

```
cd server && cp .env.example .env     # set DATABASE_URL, ADMIN_KEY
npm install && npm start              # http://localhost:3000
```

APPENDIX C

REPOSITORY STRUCTURE

```
Diya/
├── DiyaMeditation/          kiosk desktop app (C# / .NET 8 / Avalonia)
│   ├── Program.cs, App.axaml(.cs)
│   ├── Views/               MainWindow (shell), HomeView (main screen)
│   ├── Models/              VisitorApiClient, VisitorData, VisitorQr
│   ├── Services/            PipelineRunner, ReportRenderer
│   ├── scripts/             run1.sh, fullscreen-fixer.sh
│   ├── deploy/              build-deb.sh, setup-kiosk.sh, .service, .desktop
│   └── docker/              browser preview (Xvfb + noVNC)
├── server/                  Node/Express API: server.js, db.js, schema.sql
├── registration/            index.html, admin.html, scan.html
├── testkit/                 run1.mock.sh, sample-report.pdf
├── package/                 prebuilt .deb files (amd64, arm64)
├── render.yaml              hosting blueprint
└── SETUP.md, HANDOFF.md     setup guide and handoff notes
```